

SlashRing — Soluções para nível de especificação

Documento de correções para levar o plugin ao nível das especificações.

Branch: master · Baseado na leitura real do código-fonte.

Como aplicar: um item por vez, no Aider. Depois de cada item: **Ctrl+F7** no arquivo, ouça / A-B, e **commit**. Nunca aplique tudo de uma vez. Ordem recomendada: P0 → P1 → P2 → P3.

Resumo das mudanças

- **PluginProcessor.cpp** — edição pequena (3 constantes do cabinet) · P0
- **CabinetVoice.h / .cpp** — arquivo completo (cabinet vira IR pura) · P0
- **PluginProcessor.h** — edição pequena (oversampling 4x -> 8x) · P1
- **DelayModule.h / .cpp** — arquivo completo (high-cut no feedback) · P1
- **AmpModule.h / .cpp** — arquivo completo (power amp: sag, presence, resonance, 2 estagios) · P2
- **ReverbModule.h / .cpp** — arquivo completo (pre-delay, low-cut, mix real) · P2
- **Excluir:** SpeakerNonLinearity, CabinetPhysics, CabinetAcoustics, CabinetEngine (.h e .cpp) · P3
- **Limpeza:** remover diagnostico TD-010 e flag CAB-002 · P3

P0 — Fazer o cabinet soar (prioridade máxima)

O cabinet "mata o som" por dois motivos somados: (1) o mixer está configurado com `blend = 1.0`, o que — pela conta do `CabinetMixer` — manda **100% da voz B**, que NUNCA recebe IR; e (2) mesmo a voz certa passa por 3 modelagens redundantes antes da IR. P0-1 conserta o roteamento; P0-2 reduz o cabinet a IR pura (a decisão que você já tomou), o que também remove uma fonte de aliasing e corta vários arquivos.

P0-1 — Corrigir o roteamento para a IR realmente tocar

Arquivo: **Source/PluginProcessor.cpp**, dentro de `updateParameterState()`, bloco `// CABINET` (marcado como CAB-001B).

Localize estas três constantes e troque os valores:

```
// ANTES
constexpr float width      = 0.5f;    // default width
constexpr float blend      = 1.0f;    // 100% blend (voice A only)
constexpr float outputGainDb = 0.0f;   // 0 dB (unity)

// DEPOIS
constexpr float width      = 0.0f;    // centralizado, sem desbalanceio L/R
constexpr float blend      = 0.0f;    // 0.0 = 100% voz A (a que tem a IR)
constexpr float outputGainDb = 6.0f;   // makeup: compensa a lei de pan (~0.5) do mixer
```

Por quê: no `CabinetMixer::process`, `mixedA = ptrA * (1 - blend)` e `mixedB = ptrB * blend`. Com `blend = 0.0`, a saída passa a ser a voz A (que carrega a IR de fábrica). O pan law multiplica por ~ 0.5 , então `outputGainDb = 6.0` devolve o nível à unidade. Teste em A-B com volume igualado.

P0-2 — Reduzir o cabinet a IR pura

Substitua os DOIS arquivos abaixo por completo. Isso remove `SpeakerNonLinearity`, `CabinetPhysics` e `CabinetAcoustics` do caminho (a IR já captura alto-falante + caixa + microfone). `setPhysics` vira no-op só para manter a interface que o `CabinetModule` chama.

Arquivo completo: **Source/CabinetVoice.h**

```
#pragma once
#include <JuceHeader.h>
#include "CabinetDataTypes.h"
#include "CabinetConvolution.h"

class CabinetVoice final
{
public:
    CabinetVoice() = default;
    ~CabinetVoice() = default;

    void prepare (const juce::dsp::ProcessSpec& spec) noexcept;
    void reset() noexcept;

    void updateIRBuffer (CachedIRData::Ptr irData) noexcept;
    void setVoiceParameters (float gainDb, float lowCut, float highCut, float delayMs, float micDist, float micAngle) noexcept;
    void setPhysics (float cabSize, float openBack, float drive, float breakup) noexcept; // no-op (IR-only)

    void process (juce::dsp::ProcessContextReplacing<float>& context) noexcept;

private:
    double sampleRate = 44100.0;
    float gainLinear = 1.0f;
    float delaySamplesTarget = 0.0f;

    juce::dsp::AudioBlock<float> monoBlock;
    juce::AudioBuffer<float> internalMonoBuffer;

    CabinetConvolution cabinetConvolution;

    juce::dsp::ProcessorDuplicator<juce::dsp::IIR::Filter<float>, juce::dsp::IIR::Coefficients<float>> lowCutFilter;
    juce::dsp::ProcessorDuplicator<juce::dsp::IIR::Filter<float>, juce::dsp::IIR::Coefficients<float>> highCutFilter;

    juce::dsp::DelayLine<float, juce::dsp::DelayLineInterpolationTypes::Linear> microDelayLine { 4800 };

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (CabinetVoice)
```

```
};
```

Arquivo completo: Source/CabinetVoice.cpp

```
#include "CabinetVoice.h"
```

```
void CabinetVoice::prepare (const juce::dsp::ProcessSpec& spec) noexcept  
{
```

```
    sampleRate = spec.sampleRate;
```

```
    internalMonoBuffer.setSize (1, static_cast<int> (spec.maximumBlockSize));  
    monoBlock = juce::dsp::AudioBlock<float> (internalMonoBuffer);
```

```
    juce::dsp::ProcessSpec monoSpec;  
    monoSpec.sampleRate = spec.sampleRate;  
    monoSpec.maximumBlockSize = spec.maximumBlockSize;  
    monoSpec.numChannels = 1;
```

```
    cabinetConvolution.prepare (monoSpec);
```

```
    lowCutFilter.prepare (spec);  
    highCutFilter.prepare (spec);  
    microDelayLine.prepare (spec);  
    microDelayLine.setMaximumDelayInSamples (4800);
```

```
    // Defaults seguros ate setVoiceParameters ser chamado
```

```
    *lowCutFilter.state = *juce::dsp::IIR::Coefficients<float>::makeHighPass (sampleRate, 70.0f);  
    *highCutFilter.state = *juce::dsp::IIR::Coefficients<float>::makeLowPass (sampleRate, 9000.0f);
```

```
    reset();
```

```
}
```

```
void CabinetVoice::reset() noexcept
```

```
{
```

```
    cabinetConvolution.reset();  
    lowCutFilter.reset();  
    highCutFilter.reset();  
    microDelayLine.reset();  
    internalMonoBuffer.clear();
```

```
}
```

```
void CabinetVoice::updateIRBuffer (CachedIRData::Ptr irData) noexcept
```

```
{
```

```
    cabinetConvolution.updateIR (irData);
```

```
}
```

```
void CabinetVoice::setVoiceParameters (float gainDb, float lowCut, float highCut, float delayMs, float micDist, float micAngle) noexcept
```

```
{
```

```
    juce::ignoreUnused (micDist, micAngle);
```

```
    gainLinear = juce::Decibels::decibelsToGain (gainDb);  
    delaySamplesTarget = static_cast<float> ((delayMs / 1000.0) * sampleRate);
```

```
    const float clampedLow = juce::jlimit (20.0f, 600.0f, lowCut);  
    const float clampedHigh = juce::jlimit (800.0f, 20000.0f, highCut);
```

```
    *lowCutFilter.state = *juce::dsp::IIR::Coefficients<float>::makeHighPass (sampleRate, clampedLow);  
    *highCutFilter.state = *juce::dsp::IIR::Coefficients<float>::makeLowPass (sampleRate, clampedHigh);
```

```
}
```

```
void CabinetVoice::setPhysics (float, float, float, float) noexcept
```

```
{
```

```
    // IR-only: a fisica do alto-falante ja esta capturada na IR. No-op por compatibilidade.
```

```
}
```

```
void CabinetVoice::process (juce::dsp::ProcessContextReplacing<float>& context) noexcept
```

```
{
```

```
    if (context.isBypassed)  
        return;
```

```
    auto inBlock = context.getInputBlock();  
    auto outBlock = context.getOutputBlock();  
    const size_t numSamples = inBlock.getNumSamples();
```

```
    // 1) Soma para mono
```

```
    float* monoPtr = internalMonoBuffer.getWritePointer (0);  
    const float* srcL = inBlock.getChannelPointer (0);  
    const float* srcR = inBlock.getNumChannels() > 1 ? inBlock.getChannelPointer (1) : srcL;  
    for (size_t i = 0; i < numSamples; ++i)  
        monoPtr[i] = (srcL[i] + srcR[i]) * 0.5f;
```

```
    // 2) IR (convolucao) no dominio mono
```

```
    auto activeMonoBlock = monoBlock.getSubBlock (0, numSamples);
```

```

juce::dsp::ProcessContextReplacing<float> monoContext (activeMonoBlock);
cabinetConvolution.process (monoContext);

// 3) Copia mono -> todos os canais de saida
const size_t outChannels = outBlock.getNumChannels();
for (size_t ch = 0; ch < outChannels; ++ch)
    juce::FloatVectorOperations::copy (outBlock.getChannelPointer (ch), monoPtr, static_cast<int> (numSamples));

// 4) Filtros de trim (HPF / LPF)
lowCutFilter.process (context);
highCutFilter.process (context);

// 5) Micro-delay (distancia de mic) + ganho de saida
if (delaySamplesTarget > 0.5f)
{
    microDelayLine.setDelay (delaySamplesTarget);
    for (size_t ch = 0; ch < outChannels; ++ch)
    {
        float* dst = outBlock.getChannelPointer (ch);
        for (size_t s = 0; s < numSamples; ++s)
        {
            const float in = dst[s];
            microDelayLine.pushSample (static_cast<int> (ch), in);
            dst[s] = microDelayLine.popSample (static_cast<int> (ch)) * gainLinear;
        }
    }
}
else
{
    for (size_t ch = 0; ch < outChannels; ++ch)
    {
        float* dst = outBlock.getChannelPointer (ch);
        for (size_t s = 0; s < numSamples; ++s)
            dst[s] *= gainLinear;
    }
}
}

```

Depois de aplicar P0-1 e P0-2: compile, e com o cabinet ligado o timbre tem que soar como um cab Marshall de verdade (a IR entrou), sem o abafamento anterior.

P1 — Ruído e agudos

P1-1 — Aumentar o oversampling (4x -> 8x)

Arquivo: **Source/PluginProcessor.h**. Isso ataca o fizz do TD-010 vindo do amp em ganho alto. Custa CPU — aplique e verifique se compensa.

```
// ANTES
juce::dsp::Oversampling<float> oversampling
{
    2,
    2,
    juce::dsp::Oversampling<float>::filterHalfBandPolyphaseIIR
};

// DEPOIS
juce::dsp::Oversampling<float> oversampling
{
    2,
    3, // factor 3 = 8x (era 2 = 4x)
    juce::dsp::Oversampling<float>::filterHalfBandPolyphaseIIR
};
```

Observação: boa parte do fizz também some com o P0-2, porque o SpeakerNonLinearity (que rodava em base rate, sem oversampling, gerando aliasing) sai do caminho.

P1-2 — Delay com repeticoes analogicas (high-cut no feedback)

Substitua os dois arquivos. A mudança adiciona um filtro passa-baixas de um polo (~4 kHz) no caminho de realimentacao, escurecendo progressivamente as repeticoes (caráter analógico da spec).

Arquivo completo: **Source/DelayModule.h**

```
#pragma once
#include <JuceHeader.h>

class DelayModule
{
public:
    DelayModule();
    ~DelayModule();

    void prepare (double sampleRate, int samplesPerBlock, int numChannels);
    void reset();
    void process (juce::AudioBuffer<float>& buffer);

    void setDelayTime (float newDelayMs);
    void setFeedback (float newFeedback);
    void setMix (float newMix);

private:
    juce::dsp::DelayLine<float, juce::dsp::DelayLineInterpolationTypes::Linear> leftDelay { 96000 };
    juce::dsp::DelayLine<float, juce::dsp::DelayLineInterpolationTypes::Linear> rightDelay { 96000 };

    juce::SmoothedValue<float, juce::ValueSmoothingTypes::Linear> delayTimeSmoothed;
    juce::SmoothedValue<float, juce::ValueSmoothingTypes::Linear> feedbackSmoothed;
    juce::SmoothedValue<float, juce::ValueSmoothingTypes::Linear> mixSmoothed;

    // High-cut no caminho de feedback (repeticoes analogicas) — NOVO
    float fbLpfCoeff = 1.0f;
    float fbStateL = 0.0f;
    float fbStateR = 0.0f;
    static constexpr float feedbackToneHz = 4000.0f;

    float targetDelayMs = 420.0f;
    float targetFeedback = 0.35f;
    float targetMix = 0.22f;

    double currentSampleRate = 44100.0;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (DelayModule)
};
```

Arquivo completo: Source/DelayModule.cpp

```
#include "DelayModule.h"

DelayModule::DelayModule() {}
DelayModule::~DelayModule() {}

void DelayModule::prepare (double sampleRate, int samplesPerBlock, int numChannels)
{
    currentSampleRate = sampleRate;

    juce::dsp::ProcessSpec spec;
    spec.sampleRate = sampleRate;
    spec.maximumBlockSize = static_cast<juce::uint32> (samplesPerBlock);
    spec.numChannels = static_cast<juce::uint32> (numChannels);

    leftDelay.prepare (spec);
    rightDelay.prepare (spec);
    leftDelay.setMaximumDelayInSamples (static_cast<int> (sampleRate * 2.5));
    rightDelay.setMaximumDelayInSamples (static_cast<int> (sampleRate * 2.5));

    delayTimeSmoothed.reset (sampleRate, 0.02);
    feedbackSmoothed.reset (sampleRate, 0.02);
    mixSmoothed.reset (sampleRate, 0.02);
    delayTimeSmoothed.setCurrentAndTargetValue (targetDelayMs);
    feedbackSmoothed.setCurrentAndTargetValue (targetFeedback);
    mixSmoothed.setCurrentAndTargetValue (targetMix);

    // Coeficiente do LPF de feedback (one-pole)
    fbLpfCoeff = 1.0f - std::exp (-2.0f * juce::MathConstants<float>::pi * feedbackToneHz / (float) sampleRate);

    reset();
}

void DelayModule::reset()
{
    leftDelay.reset();
    rightDelay.reset();
    fbStateL = 0.0f;
    fbStateR = 0.0f;
}

void DelayModule::process (juce::AudioBuffer<float>& buffer)
{
    juce::ScopedNoDenormals noDenormals;

    const int numChannels = buffer.getNumChannels();
    const int numSamples = buffer.getNumSamples();

    for (int sample = 0; sample < numSamples; ++sample)
    {
        const float delayMs = delayTimeSmoothed.getNextValue();
        const float feedback = feedbackSmoothed.getNextValue();
        const float mix = mixSmoothed.getNextValue();

        const float delaySamples = (delayMs / 1000.0f) * static_cast<float> (currentSampleRate);
        leftDelay.setDelay (delaySamples);
        rightDelay.setDelay (delaySamples);

        for (int channel = 0; channel < numChannels; ++channel)
        {
            auto* samples = buffer.getWritePointer (channel);
            const float input = samples[sample];

            const float delayed = (channel == 0) ? leftDelay.popSample (0)
                                                : rightDelay.popSample (0);

            // High-cut one-pole no caminho de feedback (escurece as repeticoes)
            float& fbState = (channel == 0) ? fbStateL : fbStateR;
            fbState += fbLpfCoeff * (delayed - fbState);

            const float feedbackInput = input + (fbState * feedback);

            if (channel == 0) leftDelay.pushSample (0, feedbackInput);
            else rightDelay.pushSample (0, feedbackInput);

            samples[sample] = (input * (1.0f - mix)) + (delayed * mix);
        }
    }
}

void DelayModule::setDelayTime (float newDelayMs)
{
    targetDelayMs = juce::jlimit (1.0f, 2000.0f, newDelayMs);
}
```

```
        delayTimeSmoothed.setTargetValue (targetDelayMs);
    }

void DelayModule::setFeedback (float newFeedback)
{
    targetFeedback = juce::jlimit (0.0f, 0.95f, newFeedback);
    feedbackSmoothed.setTargetValue (targetFeedback);
}

void DelayModule::setMix (float newMix)
{
    targetMix = juce::jlimit (0.0f, 1.0f, newMix);
    mixSmoothed.setTargetValue (targetMix);
}
```

P2 — Qualidade (opcional, testar um de cada vez)

Estes itens adicionam DSP nova. Aplique **depois** do P1-1 (oversampling 8x) e teste isolado; se não gostar, git restore e volta.

P2-1 — Amp: power amp de verdade (sag, presence, resonance, 2 estagios)

Mantém a mesma interface pública (nada muda no PluginProcessor). Adiciona: 2º estágio de preamp, low-shelf de resonance (grave do power amp), sag (compressão dinâmica) e presence pós power amp.

Arquivo completo: Source/AmpModule.h

```
#pragma once
#include <JuceHeader.h>

class AmpModule
{
public:
    AmpModule();
    ~AmpModule();

    void prepare (double sampleRate, int samplesPerBlock, int numChannels);
    void process (juce::dsp::AudioBlock<float>& block);

    void reset();
    void release();

    void setGain (float value);
    void setBass (float value);
    void setMiddle (float value);
    void setTreble (float value);
    void setPresence (float value);
    void setMaster (float value);

    float getCurrentGainValue() const noexcept { return gainSmoothed.getCurrentValue(); }

private:
    juce::dsp::ProcessorDuplicator<juce::dsp::IIR::Filter<float>, juce::dsp::IIR::Coefficients<float>> inputHPF;
    juce::dsp::ProcessorDuplicator<juce::dsp::IIR::Filter<float>, juce::dsp::IIR::Coefficients<float>> bassFilter;
    juce::dsp::ProcessorDuplicator<juce::dsp::IIR::Filter<float>, juce::dsp::IIR::Coefficients<float>> midFilter;
    juce::dsp::ProcessorDuplicator<juce::dsp::IIR::Filter<float>, juce::dsp::IIR::Coefficients<float>> trebleFilter;
    juce::dsp::ProcessorDuplicator<juce::dsp::IIR::Filter<float>, juce::dsp::IIR::Coefficients<float>> resonanceShelf; // NOVO
    juce::dsp::ProcessorDuplicator<juce::dsp::IIR::Filter<float>, juce::dsp::IIR::Coefficients<float>> presenceShelf;
    juce::dsp::ProcessorDuplicator<juce::dsp::IIR::Filter<float>, juce::dsp::IIR::Coefficients<float>> outputLPF;

    juce::SmoothedValue<float> gainSmoothed;
    juce::SmoothedValue<float> masterSmoothed;

    // SAG (power amp) – NOVO
    float sagEnvelope = 0.0f;
    float sagAttackCoeff = 0.0f;
    float sagReleaseCoeff = 0.0f;
    static constexpr float sagAmount = 0.25f;

    float gain = 5.0f, bass = 5.0f, middle = 5.0f, treble = 5.0f, presence = 5.0f, master = 6.0f;
    double currentSampleRate = 44100.0;
    float currentDriveGain = 1.0f;
    float currentMasterGain = 1.0f;

    void updateToneStack();

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (AmpModule)
};
```

Arquivo completo: Source/AmpModule.cpp

```
#include "AmpModule.h"

AmpModule::AmpModule() {}
AmpModule::~AmpModule() {}

void AmpModule::reset()
{
    inputHPF.reset();
    bassFilter.reset();
    midFilter.reset();
```



```

    trebleFilter.reset();
    resonanceShelf.reset();
    presenceShelf.reset();
    outputLPF.reset();
    sagEnvelope = 0.0f;
}

void AmpModule::release() {}

void AmpModule::prepare (double sampleRate, int samplesPerBlock, int numChannels)
{
    currentSampleRate = sampleRate;

    gainSmoothed.reset (sampleRate, 0.02);
    masterSmoothed.reset (sampleRate, 0.02);
    gainSmoothed.setCurrentAndTargetValue (currentDriveGain);
    masterSmoothed.setCurrentAndTargetValue (currentMasterGain);

    // SAG: constantes de tempo (attack ~5 ms, release ~60 ms)
    sagAttackCoeff = 1.0f - std::exp (-1.0f / (0.005f * (float) sampleRate));
    sagReleaseCoeff = 1.0f - std::exp (-1.0f / (0.060f * (float) sampleRate));

    juce::dsp::ProcessSpec spec;
    spec.sampleRate = sampleRate;
    spec.maximumBlockSize = static_cast<juce::uint32> (samplesPerBlock);
    spec.numChannels = static_cast<juce::uint32> (numChannels);

    inputHPF.prepare (spec);
    bassFilter.prepare (spec);
    midFilter.prepare (spec);
    trebleFilter.prepare (spec);
    resonanceShelf.prepare (spec);
    presenceShelf.prepare (spec);
    outputLPF.prepare (spec);

    *inputHPF.state = *juce::dsp::IIR::Coefficients<float>::makeHighPass (sampleRate, 80.0f);
    *outputLPF.state = *juce::dsp::IIR::Coefficients<float>::makeLowPass (sampleRate, 9000.0f);

    // Resonance: voicing fixo do grave do power amp. Pode virar parametro depois.
    *resonanceShelf.state = *juce::dsp::IIR::Coefficients<float>::makeLowShelf (sampleRate, 100.0f, 0.707f, juce::Decibels::decibels);

    updateToneStack();
    reset();
}

void AmpModule::updateToneStack()
{
    const float bassGain = juce::jmap (bass, 0.0f, 10.0f, 0.5f, 2.0f);
    const float midGain = juce::jmap (middle, 0.0f, 10.0f, 0.5f, 2.0f);
    const float trebleGain = juce::jmap (treble, 0.0f, 10.0f, 0.5f, 2.0f);
    const float presenceGain = juce::jmap (presence, 0.0f, 10.0f, 0.5f, 2.0f);

    *bassFilter.state = *juce::dsp::IIR::Coefficients<float>::makeLowShelf (currentSampleRate, 120.0f, 0.707f, bassGain);
    *midFilter.state = *juce::dsp::IIR::Coefficients<float>::makePeakFilter (currentSampleRate, 750.0f, 0.8f, midGain);
    *trebleFilter.state = *juce::dsp::IIR::Coefficients<float>::makeHighShelf (currentSampleRate, 3500.0f, 0.707f, trebleGain);
    *presenceShelf.state = *juce::dsp::IIR::Coefficients<float>::makeHighShelf (currentSampleRate, 5000.0f, 0.707f, presenceGain);
}

void AmpModule::process (juce::dsp::AudioBlock<float>& block)
{
    juce::ScopedNoDenormals noDenormals;

    // INPUT HPF
    { juce::dsp::ProcessContextReplacing<float> ctx (block); inputHPF.process (ctx); }

    const auto numChannels = block.getNumChannels();
    const auto numSamples = block.getNumSamples();

    // PREAMP - 2 estagios, sample-outer / channel-inner (TD-003)
    for (size_t i = 0; i < numSamples; ++i)
    {
        const float drive = gainSmoothed.getNextValue();
        for (size_t ch = 0; ch < numChannels; ++ch)
        {
            auto* s = block.getChannelPointer (ch);
            float x = s[i] * drive;
            x = (x >= 0.0f) ? std::tanh (x * 1.75f) : std::tanh (x * 1.25f); // estagio 1
            x = (x >= 0.0f) ? std::tanh (x * 1.10f) : std::tanh (x * 0.90f); // estagio 2
            s[i] = x;
        }
    }

    // TONE STACK
    { juce::dsp::ProcessContextReplacing<float> ctx (block);
        bassFilter.process (ctx); midFilter.process (ctx); trebleFilter.process (ctx); }
}

```

```

// RESONANCE (grave do power amp)
{ juce::dsp::ProcessContextReplacing<float> ctx (block); resonanceShelf.process (ctx); }

// POWER AMP – sag + tanh assimétrico + master, sample-outer / channel-inner
for (size_t i = 0; i < numSamples; ++i)
{
    const float masterGain = masterSmoothed.getNextValue();

    float lvl = 0.0f;
    for (size_t ch = 0; ch < numChannels; ++ch)
        lvl = juce::jmax (lvl, std::abs (block.getChannelPointer (ch)[i]));

    const float coeff = (lvl > sagEnvelope) ? sagAttackCoeff : sagReleaseCoeff;
    sagEnvelope += coeff * (lvl - sagEnvelope);
    const float sagGain = 1.0f / (1.0f + sagAmount * sagEnvelope);

    for (size_t ch = 0; ch < numChannels; ++ch)
    {
        auto* s = block.getChannelPointer (ch);
        float x = s[i];
        x = (x >= 0.0f) ? std::tanh (x * 1.40f) : std::tanh (x * 1.30f);
        x *= sagGain;
        x *= masterGain;
        s[i] = x;
    }
}

// PRESENCE (agudo do power amp)
{ juce::dsp::ProcessContextReplacing<float> ctx (block); presenceShelf.process (ctx); }

// OUTPUT LPF
{ juce::dsp::ProcessContextReplacing<float> ctx (block); outputLPF.process (ctx); }
}

void AmpModule::setGain (float value)
{
    gain = juce::jlimit (0.0f, 10.0f, value);
    const float normalized = gain / 10.0f;
    currentDriveGain = juce::jmap (normalized, 1.0f, 35.0f);
    gainSmoothed.setTargetValue (currentDriveGain);
}

void AmpModule::setBass (float value) { bass = value; updateToneStack(); }
void AmpModule::setMiddle (float value) { middle = value; updateToneStack(); }
void AmpModule::setTreble (float value) { treble = value; updateToneStack(); }
void AmpModule::setPresence (float value) { presence = value; updateToneStack(); }

void AmpModule::setMaster (float value)
{
    master = juce::jlimit (0.0f, 10.0f, value);
    const float normalized = master / 10.0f;
    currentMasterGain = juce::Decibels::decibelsToGain (juce::jmap (normalized, -60.0f, 0.0f));
    masterSmoothed.setTargetValue (currentMasterGain);
}

```

P2-2 — Reverb: pre-delay, low-cut e mix real

Substituí o reverb (hoje placeholder) por um com pre-delay (~25 ms), low-cut no envio (~120 Hz) e balanço dry/wet correto (reverb 100% wet + mix manual). Também garante ScopedNoDenormals (TD-009).

Arquivo completo: Source/ReverbModule.h

```

#pragma once
#include <JuceHeader.h>

class ReverbModule
{
public:
    ReverbModule();
    ~ReverbModule();

    void prepare (double sampleRate, int samplesPerBlock, int numChannels);
    void setMix (float newMix);
    void process (juce::AudioBuffer<float>& buffer);

private:
    float mix = 0.15f;
    double currentSampleRate = 44100.0;

    juce::dsp::Reverb reverb;

```

```

// Pre-delay + low-cut no envio – NOVO
juce::dsp::DelayLine<float>, juce::dsp::DelayLineInterpolationTypes::None> preDelay { 96000 };
juce::dsp::ProcessorDuplicator<juce::dsp::IIR::Filter<float>, juce::dsp::IIR::Coefficients<float>>> sendHighPass;

juce::AudioBuffer<float> wetBuffer;

static constexpr float preDelayMs = 25.0f;
static constexpr float lowCutHz = 120.0f;

JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (ReverbModule)
};

```

Arquivo completo: Source/ReverbModule.cpp

```

#include "ReverbModule.h"

ReverbModule::ReverbModule() {}
ReverbModule::~ReverbModule() {}

void ReverbModule::prepare (double sampleRate, int samplesPerBlock, int numChannels)
{
    currentSampleRate = sampleRate;

    juce::dsp::ProcessSpec spec;
    spec.sampleRate = sampleRate;
    spec.maximumBlockSize = static_cast<juce::uint32> (samplesPerBlock);
    spec.numChannels = static_cast<juce::uint32> (numChannels);

    reverb.reset();
    reverb.prepare (spec);

    juce::dsp::Reverb::Parameters params;
    params.roomSize = 0.5f;
    params.damping = 0.5f;
    params.wetLevel = 1.0f; // 100% wet – o balanço é feito manualmente em process()
    params.dryLevel = 0.0f;
    params.width = 1.0f;
    params.freezeMode = 0.0f;
    reverb.setParameters (params);

    sendHighPass.prepare (spec);
    *sendHighPass.state = *juce::dsp::IIR::Coefficients<float>::makeHighPass (sampleRate, lowCutHz);
    sendHighPass.reset();

    preDelay.prepare (spec);
    preDelay.setMaximumDelayInSamples (static_cast<int> (sampleRate * 0.2) + 1);
    preDelay.reset();
    preDelay.setDelay (static_cast<float> ((preDelayMs / 1000.0) * sampleRate));

    wetBuffer.setSize (numChannels, samplesPerBlock);
}

void ReverbModule::setMix (float newMix)
{
    mix = juce::jlimit (0.0f, 1.0f, newMix);
}

void ReverbModule::process (juce::AudioBuffer<float>& buffer)
{
    juce::ScopedNoDenormals noDenormals;

    const int numChannels = buffer.getNumChannels();
    const int numSamples = buffer.getNumSamples();

    // Copia seca -> envio (wet)
    for (int ch = 0; ch < numChannels; ++ch)
        wetBuffer.copyFrom (ch, 0, buffer, ch, 0, numSamples);

    juce::dsp::AudioBlock<float> wetBlock (wetBuffer);
    auto wetSub = wetBlock.getSubBlock (0, static_cast<size_t> (numSamples));

    // Low-cut no envio (mantem o grave seco)
    { juce::dsp::ProcessContextReplacing<float> ctx (wetSub); sendHighPass.process (ctx); }

    // Pre-delay
    for (int ch = 0; ch < numChannels; ++ch)
    {
        float* w = wetBuffer.getWritePointer (ch);
        for (int s = 0; s < numSamples; ++s)
        {
            preDelay.pushSample (ch, w[s]);
            w[s] = preDelay.popSample (ch);
        }
    }
}

```

```
}

// Reverb 100% wet
{ juce::dsp::ProcessContextReplacing<float> ctx (wetSub); reverb.process (ctx); }

// Mix dry / wet
for (int ch = 0; ch < numChannels; ++ch)
{
    float* dry = buffer.getWritePointer (ch);
    const float* wet = wetBuffer.getReadPointer (ch);
    for (int s = 0; s < numSamples; ++s)
        dry[s] = dry[s] * (1.0f - mix) + wet[s] * mix;
}
}
```

P3 — Exclusões e limpeza

Arquivos para EXCLUIR do projeto

Depois de aplicar o P0-2 (CabinetVoice IR-only), estes arquivos ficam sem uso e devem ser removidos:

- Source/SpeakerNonLinearity.h e Source/SpeakerNonLinearity.cpp
- Source/CabinetPhysics.h e Source/CabinetPhysics.cpp
- Source/CabinetAcoustics.h e Source/CabinetAcoustics.cpp
- Source/CabinetEngine.h e Source/CabinetEngine.cpp (codigo morto: nao e usado pelo CabinetModule nem pelo PluginProcessor)

Como excluir com segurança (Projucer):

1. Abra o SlashRing.jucer no Projucer.
2. Na árvore de arquivos (aba Files), selecione cada um dos arquivos acima e remova (Delete / Remove File).
3. Salve o projeto no Projucer (isso regenera o build sem esses arquivos).
4. Só então apague os arquivos do disco.
5. Confirme que nada mais os inclui: o novo CabinetVoice.h não os inclui mais, e o CabinetModule só inclui CabinetVoice.h, CabinetMixer.h e IRLoader.h. Faça um build completo (F7) para validar.

Verificacao rapida antes de apagar: procure no projeto por `#include "SpeakerNonLinearity.h"`, `#include "CabinetPhysics.h"`, `#include "CabinetAcoustics.h"` e `#include "CabinetEngine.h"`. Depois do P0-2, não deve sobrar nenhuma referência.

Limpeza de código (nao sao arquivos, sao trechos)

Diagnostico TD-010 em **Source/PluginProcessor.cpp** — remover (é temporário e polui o hot path):

- O bloco namespace { ... } no topo com `td010ComputeRmsPeakDb` e `td010AppendDiagLogLine`.
- Os três blocos marcados `// TD-010 GAIN MAP B — TEMPORARY DIAGNOSTIC — ... TAP — BEGIN/END` dentro do `processBlock`.
- O bloco grande `// TD-010 ... PARAMETER-STATE + LOG — BEGIN/END`.

Em **Source/PluginProcessor.h** — remover o membro `double td010DiagAccumSamples = 0.0;` e seu comentário.

(Os getters `getCurrentGainValue` / `getCurrentDriveValue` podem ficar; são inofensivos.)

Flag CAB-002 em **Source/CabinetModule.cpp** — simplificar `process()`: remover `constexpr bool kCabIsolation` e o `if (kCabIsolation) { ... } else { ... }`, deixando direto:

```
voiceA.process (ctxA);
voiceB.process (ctxB);
mixer.process (activeBlockA, activeBlockB, outBlock);
```

(Opcional: como com `blend = 0.0` a voz B fica inaudível, você pode nem processar a voz B para poupar CPU. Deixe para um passo separado, se quiser.)

Ordem de aplicacao (checklist)

1. P0-1 — 3 constantes no PluginProcessor.cpp -> Ctrl+F7 -> ouvir (IR entrou?) -> commit
2. P0-2 — CabinetVoice.h + .cpp -> Ctrl+F7 -> ouvir (limpo?) -> commit
3. P1-1 — oversampling 8x -> build -> ouvir (menos fizz?) -> commit
4. P1-2 — DelayModule.h + .cpp -> Ctrl+F7 -> testar repeticoes -> commit
5. P2-1 — AmpModule.h + .cpp -> Ctrl+F7 -> A-B do amp -> commit
6. P2-2 — ReverbModule.h + .cpp -> Ctrl+F7 -> A-B do reverb -> commit
7. P3 — excluir arquivos no Projucer + limpar diagnosticos -> build completo (F7) -> commit

Regra de ouro: um item, um build, um commit. Se algo piorar, `git restore .` e volte ao ultimo commit bom.